Modèle de projet — site communautaire

Blueprint réutilisable tiré de **Poker Winners** (v0.6.0)

MODÈLE v1
30/10/2026
Sans mot de passe ni secret

> **But :** reproduire un site vitrine + communauté (actualités, forum, vidéos, cours, calendrier, comptes, RGPD, boutique en option) sur un thème quelconque, de la spec au déploiement sur un mutualisé cPanel (o2switch), en ~10 étapes vérifiables. Remplacer « poker » par le thème du nouveau projet.

1. Pile technique retenue

Couche	Choix	Pourquoi / remarque
API	Symfony 7.4 LTS, PHP 8.2+ (8.3 en prod), Doctrine ORM 3	Monolithe API JSON + back-office dans le même projet
Auth	lexik/jwt (1 h) + gesdinet refresh token (7 j, usage unique)	Front SPA sans cookie de session ; refresh automatique par intercepteur
Back-office	EasyAdmin 5 (session + formulaire de connexion)	CRUD générés ; rôles USER < MODERATOR < ADMIN
Front	Angular 22 standalone, signals, `input()`, control flow, lazy routes	Toujours démarrer sur la dernière version maintenue
Base	MariaDB 10.6+ partout (Docker en local, MariaDB 11.4 en prod)	PostgreSQL d'o2switch trop ancien (9.6) pour Doctrine DBAL 4
Local	Docker : MariaDB (3307), Mailhog (1025/8025) ; API `php -S localhost:8000` ; front `ng serve --port 4300`	Ports distincts par projet pour en faire tourner plusieurs
Tests	PHPUnit fonctionnels (SQLite via `.env.test`), Jasmine/Karma ChromeHeadless	46 + 9 tests à v0.6.0
Hébergement	o2switch mutualisé (cPanel, uapi, MultiPHP, Let's Encrypt)	Front statique + API sur sous-domaine

2. Arborescence du dépôt

```
/CLAUDE.md                règles du projet (lancement, tests, interdits)
/CHANGELOG.md  /docs/ROADMAP.md     versionnage SemVer + feuille de route
/docker-compose.yml       MariaDB, Mailhog (nommer le projet pour des volumes stables)
/backPoker   (API Symfony)
src/{Entity,Enum,Repository,Dto,Service,Command,Controller/Api,Controller/Admin,Content,Migrations}
/frontPoker  (Angular)      src/app/{core/api,core/auth,core/seo,shared/components,features/*}
public/{.htaccess,robots.txt}
/deploy      server-setup.sh · deploy.sh · update.sh · build-front.sh · publish-front.sh (variante SSH)
/docs        actualites/*.json · calendrier/*.json · videos/ · design/ · deploiement/ · recap/
```

3. Modèle de données (12 entités)

Entité	Rôle
User · ConsentLog · RefreshToken	Compte (email nullable pour l'anonymisation, pseudo, rôles, isVerified, jetons de vérif/suppression), preuves de consentement (version CGU, IP, date), refresh JWT
Article (enum ArticleScope)	Actualités par portée (ex. france / international / live / online), sources en liens nofollow
Video (enum VideoLevel) · VideoFavorite	Vidéos YouTube sous licence CC BY + favoris par membre
ForumCategory · ForumTopic · ForumPost · ForumSubscription · ForumPostReport (enum ReportReason)	Forum modéré, abonnements, signalements avec file de modération
Tournament (enum TournamentZone)	Calendrier d'événements par zone, unique (nom, date de début)
v2 : Product · Order · OrderItem	Boutique (TVA par catégorie, prix en centimes, snapshot des lignes), derrière un drapeau

Leçons et quiz : données TypeScript côté front (`features/lessons/data/*.lessons.ts`), 3 niveaux × 15 leçons × quiz de 5 questions — pas de table tant que la progression reste locale.

4. API (préfixe /api)

Routes publiques	Routes connectées (JWT)
`POST login` , `token/refresh` , `register` , `verify-email` , `verify-email/resend`	`GET/PATCH me` , `PATCH me/password` , `GET me/export` , `DELETE me` → email de confirmation → `account-deletion/confirm`
`GET articles?scope=` , `articles/{slug}` , `videos` , `videos/{id}` , `tournaments?zone=` , `stats`	`me/favorites` , favoris vidéo, créer sujet / message, abonnements, `POST forum/posts/{id}/report`
`GET forum/…` (catégories, sujets, messages), `search?q=` , `members/{pseudo}`	Doc OpenAPI (Nelmio) sur `/api/doc` en **dev uniquement**

Conventions : DTO `#[MapRequestPayload]` + contraintes Validator (messages en français), réponses JSON, pagination commune, rate limiters nommés (`register, verification_resend, forum_post, forum_report, search`) + login throttling (429).

5. Sécurité et RGPD — à reprendre tels quels

Sécurité

- Secrets jamais versionnés (`.env.local` , `.env.*.local` , `*.pem`) ; générés **sur le serveur** par le script d'installation (droits 600).
- `.htaccess` : HTTPS forcé, HSTS, CSP stricte (`script-src 'self'`), X-Frame-Options DENY, fichiers sensibles bloqués.
- Angular : `optimization.styles.inlineCritical: false` (sinon un `onload` inline viole la CSP) ; copier `.htaccess` via un glob d'assets explicite.
- CORS limité aux origines du site ; `trusted_hosts` ; cookies session secure/httponly/samesite.
- Mots de passe : `PasswordStrength` niveau moyen ; bêta protégée par Basic Auth + `noindex` .

RGPD

- Consentement CGU/confidentialité journalisé (version, IP, date).
- Vérification d'email obligatoire avant de poster.
- Export JSON des données (`me/export`).
- Suppression confirmée par email puis **anonymisation** (les messages restent, l'auteur devient « Utilisateur supprimé »).
- Cron `app:gdpr:cleanup` (comptes non vérifiés, jetons expirés).
- Pages légales avec champs `[À COMPLÉTER]` centralisés dans `legal.config.ts` .
- Boutique : commandes conservées (obligation comptable) mais détachées du compte anonymisé.

- Vidéos : uniquement CC BY vérifiées, attribution ;
  lecture au clic via youtube-nocookie (sous-titres FR : `cc_load_policy=1&cc_lang_pref=fr`).

6. Contenus et outils

- **Importeurs idempotents** JSON → base : `app:articles:import`, `app:tournaments:import` (rejet des URL non http(s), dates invalides).
- **Contenu de base en prod** sans compte de démo : `app:content:seed` (classe partagée `App\Content\DefaultContent` utilisée aussi par les fixtures).
- **Fixtures** (dev) : comptes démo + forum + contenus. `app:user:promote email --role=ROLE_ADMIN`.
- **Actualités réelles** : chaque info vérifiée sur 2 sources, sources citées ; aucun contenu copié de créateurs (droit d'auteur).
- **PDF** : HTML imprimé avec Edge headless (`--print-to-pdf`) depuis un chemin sans espaces ; contrôle avec `pdfinfo` / `pdftoppm` (poppler).

7. Déploiement o2switch (sans SSH)

#	Étape	Commande / action
1	PHP 8.3 + Composer dans le compte (CLI par défaut = 8.1)	`ln -sf /opt/cpanel/ea-php83/root/usr/bin/php ~/bin/php` puis installeur Composer dans `~/bin`, PATH dans `.bashrc`
2	Cloner le dépôt	`git clone https://github.com/<compte>/<depot>.git ~/<app>`
3	Installation unique	`bash ~/<app>/deploy/server-setup.sh` : base + utilisateur aux droits limités, boîte no-reply, sous-domaines `api.` / `beta.`, `.env.local`, clés JWT, accès bêta
4	cPanel	MultiPHP Manager : `api.` et `beta.` en PHP 8.3 ; Let's Encrypt ; vérifier SPF/DKIM/DMARC
5	Contenus	`php bin/console app:content:seed --env=prod`
6	Front (sur le PC)	`bash deploy/build-front.sh` → branche `front-beta` (build + protection bêta)
7	Mise à jour (serveur)	`bash ~/<app>/deploy/update.sh` : API (composer, migrations, cache, audit) + front, sauvegarde `.previous`
8	Admin	S'inscrire, confirmer l'email, `app:user:promote … --env=prod`
9	Accès testeurs	Une ligne par testeur dans `~/.htpasswds/<app>-beta` via `openssl passwd -apr1 -stdin` (`>>`), droits 644
10	v1.0 publique	`build-front.sh --public` + `update.sh --public` — **uniquement après confirmation du propriétaire**

8. Pièges rencontrés et solutions

Symptôme	Cause	Solution
SSH « Connection timed out »	IP non retenue dans le pare-feu o2switch (la clé n'est pas en cause)	Terminal cPanel + `update.sh` ; ne jamais regénérer de clé pour un timeout
API 500 « requires PHP >= 8.2 »	Sous-domaine resté en PHP 8.1	MultiPHP Manager → 8.3
Bêta 500 après saisie du mot de passe	Fichier htpasswd illisible / binaire `htpasswd` absent	`openssl passwd -apr1` , `chmod 755 ~/.htpasswds` , `644` sur le fichier
« La page de connexion se réinitialise »	Aucun compte / compte non vérifié	Lister les comptes, confirmer à la main si l'email tarde
Email non reçu	Adresse mal saisie (rejet 550) ou spam	« Suivi de la livraison » cPanel ; recherche `from:no-reply@…`
npm « edgesOut » / conflits de peers	node_modules d'une version précédente	Supprimer node_modules + lock, `npx -y npm@11 install`
Build EBUSY sur `.htaccess`	Verrou OneDrive	Seconde tentative automatique dans `build-front.sh`
CSP bloque l'appli	CSS critique inliné par Angular	`inlineCritical: false`
API qui ne répond plus en local	Docker Desktop arrêté après redémarrage	Démarrer Docker puis `docker compose up -d`

9. Méthode de travail (avec Claude Code)

- **Sprints** : 0 socle → 1 comptes/RGPD → 2 contenus → 3 forum → 4 profil/légal → 5 finitions/déploiement ; versions SemVer (tag + CHANGELOG à chaque livraison).
- **Agents en parallèle**, chacun sur une zone distincte (back / front / contenus) ; fonctionnalités futures sur une **branche + worktree** séparés et un drapeau désactivé (ex. boutique v2).
- Consigne aux agents : « expert du domaine » (ex. expert en poker), lecture minimale, rapport court ; vérifier chaque livraison (tests, build, contrôle des sources/licences).
- Commit + push à chaque étape verte ; aucun secret dans le chat ni dans Git ; jamais de mise en ligne publique sans confirmation.
- Un `CLAUDE.md` par projet et des dossiers séparés pour ne pas mélanger deux projets.

10. Check-list pour dupliquer

1. Choisir thème, nom, domaine ; créer le dépôt GitHub et le dossier local (chemin sans espaces de préférence).
2. Copier la structure (sections 2–6), renommer les préfixes (`backXxx` , `frontXxx`), adapter les enums de portée/zone/niveau au thème.
3. Rédiger CLAUDE.md, ROADMAP, design (couleurs, police) ; choisir des ports locaux libres.
4. Produire les contenus vérifiés (JSON d'actualités, calendrier, vidéos sous licence libre, leçons).
5. Tests verts → tag v0.x → déploiement bêta (section 7) → testeurs → mentions légales → confirmation → v1.0.

Modèle établi à partir de Poker Winners v0.6.0 (dépôt github.com/redkards/poker-vitrine, tag v0.6.0) · aucun mot de passe, clé ni secret dans ce document.